

NAME: Ayaan Shaikh
Mobile no : 8055733707
Internship : Data Analytics using Python
Submission date: 18/05/2026

Task 1 : Collect data from different sources (CSV, Excel, Databases, APIs).

Objective

To gather data from multiple structured and unstructured sources, ensuring a rich and diverse dataset. This helps in improving the quality of analysis by combining information from different formats and systems.

1. Reading CSV File.

Code

```
import pandas as pd
df=pd.read_csv('User_Data_Association.csv')
print(df.head())
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

I imported the **Pandas** library to work with data in Python. Then, I loaded the dataset named **User_Data_Association.csv** into a DataFrame using the `read_csv()` function.

After successfully loading the data, I used the `head()` function to display the first five rows of the dataset .

2. Reading Excel File.

Code

```
data=pd.read_excel('Js_Mart_Data.xlsx')
print(data.head())
```

	Order ID	Order Date	Customer Name	City	Country
0	BN-2011-7407039	2017-01-01	Ruby Patel	Stockholm	Sweden
1	AZ-2011-9050313	2017-01-03	Summer Hayward	Southport	United Kingdom
2	AZ-2011-6674300	2017-01-04	Devin Huddleston	Valence	France
3	BN-2011-2819714	2017-01-04	Mary Parker	Birmingham	United Kingdom
4	AZ-2011-617423	2017-01-05	Daniel Burke	Echirolles	France

	State	Region	Segment	Ship Date	Ship Mode
0	Stockholm	North	Home Office	2017-01-05	Economy Plus
1	England	North	Consumer	2017-01-07	Economy
2	Auvergne-Rhône-Alpes	Central	Consumer	2017-01-08	Economy
3	England	North	Corporate	2017-01-09	Economy
4	Auvergne-Rhône-Alpes	Central	Home Office	2017-01-07	Priority

	lon	lat
0	18.068581	59.329324
1	-3.010113	53.645708
2	4.892360	44.933393
3	-1.890401	52.486243
4	5.718034	45.142151

I used the **Pandas** library to read data from an Excel file named **Js_Mart_Data.xlsx** using the `read_excel()` function.

After loading the data into a DataFrame, I applied the `head()` function to display the first five rows.

This allowed me to quickly inspect the dataset, understand its structure, and view sample records along with column names.

3. Reading SQL Database.

Code

```
import sqlite3
import pandas as pd

conn=sqlite3.connect('shopping.db')
cursor=conn.cursor()
cursor.execute(''' CREATE TABLE IF NOT EXISTS customer(id INTEGER PRIMARY KEY,name TEXT,age INTEGER)''')
cursor.executemany("INSERT INTO customer VALUES (?, ?, ?)", [
    (100,'HARSH',22),
    (200,'Pritam',21),
    (300,'Amit',24),
    (410,'Sneha',23),
    (500,'Rahul',20)
])

conn.commit()
print("Database created successfully!")

df=pd.read_sql("SELECT * FROM customer",conn)
print(df)

conn.close()

Database created successfully!
   id  name  age
0  100  HARSH   22
1  200  Pritam  21
2  300   Amit  24
3  410  Sneha  23
4  500  Rahul  20
```

I imported the **sqlite3** and **Pandas** libraries to work with a database and analyze data. First, I created a connection to a SQLite database named **shopping.db**.

Then, I created a table called **customer** with columns for id, name, and age (if it did not already exist).

Next, I inserted multiple records into the table using the `executemany()` function and saved the changes using `commit()`.

After successfully creating and populating the database, I retrieved all the records from the **customer** table using an SQL query (`SELECT * FROM customer`) and loaded the result into a Pandas DataFrame.

Task 3 : Filter the data and select only the required information .

Objective

To extract relevant data by applying conditions and filters, reducing noise and improving processing efficiency. This ensures that only meaningful data is used for analysis.

```
: import pandas as pd
data={'Name':      ['Rahul', 'Priya', 'Amit', 'Sneha', 'Ravi', 'Neha', 'Suresh', 'Kavya'],
      'Age':       [20, 22, 21, 23, 20, 24, 22, 21],
      'City':      ['Pune', 'Mumbai', 'Nagpur', 'Pune', 'Nashik', 'Mumbai', 'Pune', 'Nagpur'],
      'Marks':     [85.5, 90.0, 78.3, 88.7, 92.1, 76.4, 88.0, 95.2],
      'Department': ['CS', 'IT', 'CS', 'Mech', 'IT', 'CS', 'Mech', 'IT']}

df=pd.DataFrame(data)
df.to_csv('student.csv')
print("Csv File Created")

Csv File Created
```

1. Selected the required column.

```
# Selecting the required Column
col=df[['Name', 'City']]
print(col)
```

	Name	City
0	Rahul	Pune
1	Priya	Mumbai
2	Amit	Nagpur
3	Sneha	Pune
4	Ravi	Nashik
5	Neha	Mumbai
6	Suresh	Pune
7	Kavya	Nagpur

I selected specific columns (**Name** and **City**) from the DataFrame using double brackets and stored them in a new variable `col`. Then, I printed the result to view only those selected columns from the dataset.

2. Filtering data using certain conditions.

```
: # Filtering marks greater 80 from the dataset
marks=df[df['Marks'] > 80]
print(marks)
```

	Name	Age	City	Marks	Department
0	Rahul	20	Pune	85.5	CS
1	Priya	22	Mumbai	90.0	IT
3	Sneha	23	Pune	88.7	Mech
4	Ravi	20	Nashik	92.1	IT
6	Suresh	22	Pune	88.0	Mech
7	Kavya	21	Nagpur	95.2	IT

I filtered the DataFrame to select only those rows where the **Marks** column is greater than 80.

Then, I stored the filtered data in a variable `marks` and printed it to view students who scored above 80.

Task 4 : Identify errors in the data (missing values, duplicates, incorrect formats) .

Objective:

To detect inconsistencies and errors in the dataset that may lead to incorrect results. Identifying such issues helps maintain data integrity and reliability.

```
import pandas as pd

data = {
    'Name': ['Ayaan', 'Rahul', 'Sneha', 'Rahul', 'Amit'],
    'Marks': [85, 90, None, 90, 'eighty'],
    'City': ['Mumbai', 'Delhi', 'Mumbai', 'Delhi', 'Pune']
}

df = pd.DataFrame(data)

print("Missing Values:\n", df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())
print("\nData Types:\n")
print(df.info())
print("\nUnique Values in Marks:\n", df['Marks'].unique())
```

Output :

```
Missing Values:
Name      0
Marks     1
City      0
dtype: int64

Duplicate Rows: 1

Data Types:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   Name    5 non-null        object
1   Marks   4 non-null        object
2   City    5 non-null        object
dtypes: object(3)
memory usage: 252.0+ bytes
None

Unique Values in Marks:
[85 90 None 'eighty']
```

I created a sample dataset using Pandas and then checked for data errors in a single program.

I identified **missing values** using `isnull()`, detected **duplicate rows** using `duplicated()`, examined **data types** using `info()`, and found **inconsistent values** in the *Marks* column using `unique()`.

This helped me quickly understand the data quality issues in the dataset.

Task 5: Find duplicate rows in the dataset .

Objective

To locate and handle duplicate records that may skew analysis results. Removing duplicates ensures accurate calculations and prevents bias in data interpretation. By detecting duplicates using appropriate techniques and removing or handling them carefully, we ensure that each data entry is unique and valid.

```

import pandas as pd
import numpy as np

data = {
    "id": [1,2,3,4,5,6,7,8,9,3,7,1],
    "name": [
        "Ayaan", "Sara", "John", "Ali", "Meena", "Ravi",
        "Priya", "Kiran", "Rahul",
        "John", "Priya", "Ayaan"
    ],
}
df=pd.DataFrame(data)
print("Removing Duplicate rows")
print(df)

print("After Eleminating Duplicate Rows")

print(df.drop_duplicates())

```

Output :

```

Removing Duplicate rows
   id  name
0   1  Ayaan
1   2   Sara
2   3   John
3   4    Ali
4   5  Meena
5   6   Ravi
6   7  Priya
7   8  Kiran
8   9  Rahul
9   3   John
10  7  Priya
11  1  Ayaan
After Eleminating Duplicate Rows
   id  name
0   1  Ayaan
1   2   Sara
2   3   John
3   4    Ali
4   5  Meena
5   6   Ravi
6   7  Priya
7   8  Kiran
8   9  Rahul

```

I created a dataset using Pandas that contains duplicate records. First, I displayed the original data to show all entries, including duplicates.

Then, I used the `drop_duplicates()` function to remove repeated rows from the dataset.

Finally, I printed the cleaned data to show only unique records, ensuring better data quality and accurate analysis.

Task 6: Use Pandas for data cleaning .

Objective

To utilize powerful data manipulation tools in Pandas for cleaning, transforming, and preparing data efficiently. This step is essential for making the dataset analysis-ready.

```
[1]: import pandas as pd

df = pd.read_csv("iris.csv")

print("Original Data:\n", df.head())

print("\nMissing Values:\n", df.isnull().sum())

df = df.drop_duplicates()

df.columns = df.columns.str.lower()

print("\nCleaned Data Info:\n")
print(df.info())

print("\nStatistical Summary:\n", df.describe())
```

Original Data:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Output :

```
Missing Values:
  sepal_length  0
  sepal_width  0
  petal_length  0
  petal_width  0
  species      0
dtype: int64

Cleaned Data Info:

<class 'pandas.core.frame.DataFrame'>
Index: 149 entries, 0 to 149
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   sepal_length 149 non-null    float64
1   sepal_width  149 non-null    float64
2   petal_length 149 non-null    float64
3   petal_width  149 non-null    float64
4   species      149 non-null    object
dtypes: float64(4), object(1)
memory usage: 7.0+ KB
None
```

I loaded the Iris dataset using Pandas and displayed the first few rows to understand its structure.

Then, I checked for missing values to identify any incomplete data. After that, I removed duplicate rows to ensure data accuracy and standardized the column names by converting them to lowercase.

Finally, I used `info()` and `describe()` to review the cleaned dataset and understand its structure and statistical summary .

Task 7 : Correct column names and labels if they are incorrect .

Objective

To standardize column names and labels for better readability, consistency, and ease of use during analysis and coding.

```
import pandas as pd
import numpy as np

data = {
    'Name':      ['Rahul', 'Priya', 'Amit', 'Rahul', 'Ravi', 'Neha', None],
    'Age':       [20,      22,      None,  20,      20,      24,      21],
    'City':      ['Pune',  'Mumbai', 'Nagpur', 'Pune',  'Nashik', 'Mumbai', 'Pune'],
    'Marks':     [85.5,    90.0,    78.3,   85.5,    92.1,    'N/A',   88.0],
    'Date':      ['2024-01-15', '2024/02/20', '15-03-2024', '2024-01-15', '2024-04-10', '2024-05-05', '2024-06-01']
}

df = pd.DataFrame(data)
```

Output :

```
: # Column Names to Lowercase

df.columns=df.columns.str.lower()
print(df)
```

	name	age	city	marks	date
0	Rahul	20.0	Pune	85.5	2024-01-15
1	Priya	22.0	Mumbai	90.0	2024/02/20
2	Amit	NaN	Nagpur	78.3	15-03-2024
3	Rahul	20.0	Pune	85.5	2024-01-15
4	Ravi	20.0	Nashik	92.1	2024-04-10
5	Neha	24.0	Mumbai	N/A	2024-05-05
6	None	21.0	Pune	88.0	2024-06-01


```
: # Rename Column Names

df=df.rename(columns={'name':'FName','date':'Date'})
print(df)
```

	FName	age	city	marks	Date
0	Rahul	20.0	Pune	85.5	2024-01-15
1	Priya	22.0	Mumbai	90.0	2024/02/20
2	Amit	NaN	Nagpur	78.3	15-03-2024
3	Rahul	20.0	Pune	85.5	2024-01-15
4	Ravi	20.0	Nashik	92.1	2024-04-10
5	Neha	24.0	Mumbai	N/A	2024-05-05
6	None	21.0	Pune	88.0	2024-06-01

I first converted all column names to lowercase using `str.lower()` to maintain consistency. Then, I displayed the dataset to verify the changes.

After that, I renamed specific columns (e.g., name to FName and date to Date) using the `rename()` function, and printed the updated dataset to confirm the modifications.

Task 8 : Convert data into different formats (e.g., date format) .

Objective

To transform data types into appropriate formats (such as converting strings to dates or numbers), enabling accurate computations and compatibility with analysis tools.

```
# Convert Date Format
df['Date']=pd.to_datetime(df['Date'],errors='coerce')
print(df)
```

	FName	age	city	marks	Date
0	Rahul	20.0	NaN	85.5	2024-01-15
1	Priya	22.0	NaN	90.0	NaT
2	Amit	NaN	NaN	78.3	NaT
3	Rahul	20.0	NaN	85.5	2024-01-15
4	Ravi	20.0	NaN	92.1	2024-04-10
5	Neha	24.0	NaN	NaN	2024-05-05
6	None	21.0	NaN	88.0	2024-06-01

I converted the Date column into a proper datetime format using `pd.to_datetime()`.

I used `errors='coerce'` to handle invalid or incorrect date values by converting them into NaT (missing values). Finally, I printed the dataset to verify the changes.

Task 9 : Use `describe()` and `info()` functions for summary .

Objective

To gain a quick overview of the dataset, including statistical measures, data types, and non-null counts. This helps in understanding the structure and quality of the data.

info() :

```
# Info()
print(df.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   Unnamed: 0    15 non-null    int64  
1   id            15 non-null    int64  
2   name          15 non-null    object  
3   age           15 non-null    int64  
4   city          15 non-null    object  
5   salary        15 non-null    int64  
dtypes: int64(4), object(2)
memory usage: 852.0+ bytes
None
```

I applied the `info()` function on the `data.csv` dataset to get a quick summary of its structure, including column names, data types, and non-null values.

This helped me understand the dataset and identify any missing or incorrect data.

describe () :

```
# describe()
print(df.describe())
```

	Unnamed: 0	id	age	salary
count	15.000000	15.000000	15.000000	15.000000
mean	7.000000	5.933333	28.800000	58400.000000
std	4.472136	3.594970	4.61674	9796.500833
min	0.000000	1.000000	22.000000	45000.000000
25%	3.500000	3.000000	25.500000	50500.000000
50%	7.000000	6.000000	28.000000	55000.000000
75%	10.500000	8.500000	30.500000	65000.000000
max	14.000000	12.000000	40.000000	80000.000000

I used the describe() function on the data.csv dataset to generate statistical summaries such as mean, standard deviation, minimum, maximum, and quartile values.

This helped me understand the distribution and key characteristics of the numerical data in the dataset.

Task 10 : Identify missing or inconsistent information in columns .

Objective

To detect null values, anomalies, or inconsistent entries in columns and take appropriate actions such as imputation or removal, ensuring high-quality data for analysis.

```
import pandas as pd

df=pd.read_csv('data.csv')
print(df.head())
```

	Unnamed: 0	id	name	age	city	salary
0	0	1	Ayaan	25	Mumbai	50000
1	1	2	Sara	30	Pune	60000
2	2	3	John	28	Delhi	55000
3	3	4	Ali	22	Mumbai	45000
4	4	5	Meena	27	Chennai	52000

```
# Missing and Incorrect Values

print(df.isnull().sum())
```

Unnamed: 0	0
id	0
name	0
age	0
city	0
salary	0
dtype: int64	

```
: # Total Missing Values
print(df.isnull().sum().sum
      ())

0

: print(df[df.isnull().any(axis=1)])

Empty DataFrame
Columns: [Unnamed: 0, id, name, age, city, salary]
Index: []

: print(df.dtypes)

Unnamed: 0    int64
id           int64
name         object
age          int64
city         object
salary       int64
dtype: object
```

I loaded the **data.csv** dataset using Pandas and displayed the first few rows using `head()` to understand its structure. Then, I checked for missing or incorrect values using `isnull().sum()`. The result showed that all columns have zero missing values, confirming that the dataset is clean and complete.

I checked the dataset for missing values by calculating the total number of null entries and found that there are no missing values. Then, I filtered the dataset to display rows containing any null values, which returned an empty DataFrame, confirming that the data is complete.

After that, I examined the data types of each column using `dtypes`, which helped me understand the structure of the dataset and verify that all columns have appropriate data types.